

Docker Cheat Sheet & Advanced Concepts

A comprehensive guide to Docker commands, architecture, networking, and best practices.

1. Architecture & Internals

The Docker Engine: The core software that runs and manages containers. It consists of the Daemon (dockerd), the API interface, and the CLI client.

Containerd: An industry-standard container runtime. Docker uses containerd to manage the container lifecycle (pulling images, stopping/starting). It sits between the Docker Daemon and runc.

Runc: The lightweight CLI tool for spawning and running containers according to the OCI specification. It interacts directly with the Kernel.

Namespaces (Isolation): How Docker isolates processes.

- *PID*: Process isolation.
- *NET*: Network stacks.
- *MNT*: Filesystem mount points.
- *UTS*: Hostname isolation.

Cgroups (Control Groups - Limits): How Docker limits resources (CPU, RAM) so one container cannot exhaust the host.

2. Essential Lifecycle Commands

Command	Description
<code>docker run -d --name <name> <image></code>	Start a new container in detached mode.
<code>docker ps -a</code>	List all containers (running and stopped).
<code>docker stop <container_id></code>	Gracefully stop (SIGTERM).
<code>docker kill <container_id></code>	Forcefully stop (SIGKILL).
<code>docker rm <container_id></code>	Remove a stopped container.
<code>docker rmi <image_id></code>	Remove an image.
<code>docker exec -it <container> bash</code>	Open an interactive shell inside a running container.
<code>docker logs -f <container></code>	Tail the logs of a container.
<code>docker inspect <object></code>	View low-level JSON details (IP, volumes, config).

3. Dockerfile Instructions & Best Practices

CMD vs ENTRYPOINT:

CMD: Provides default arguments. Can be overridden from the CLI.

Example: `CMD ["echo", "hello"]` -> User runs `docker run image` -> Output: hello. User runs `docker`

run image world` -> Output: world.

ENTRYPOINT: The main executable. Cannot be overridden easily. Arguments passed to `docker run` are appended to it.

Use Case: Converting a container into an executable binary.

COPY vs ADD:

COPY: Copies local files to the container. Preferred for transparency.

ADD: Like COPY, but can also extract `.tar` files automatically and download files from URLs. (Avoid using for remote URLs due to image size bloating).

Multi-Stage Builds:

Used to keep image sizes small. Build the app in a 'builder' stage with all compilers (Maven, Go, Node), then copy only the binary/artifact to a final lightweight 'runtime' stage (like Alpine).

```
# Example FROM golang:1.16 AS builder WORKDIR /app COPY . . RUN go build -o myapp FROM alpine:latest COPY --from=builder /app/myapp /myapp CMD ["/myapp"]
```

4. Storage & Persistence

Containers are ephemeral. When they die, data is lost. Use these for persistence:

1. Volumes (Recommended): Managed by Docker (`/var/lib/docker/volumes`). Independent of container lifecycle. Best for DB data.

```
Cmd: `docker volume create my-vol`  
Usage: `-v my-vol:/app/data`
```

2. Bind Mounts: Maps a specific file/folder on the Host OS to the Container. Great for development (code changes reflect instantly).

```
Usage: `-v /home/user/project:/app`
```

3. tmpfs Mounts: Stored in the host system's memory only. Never written to disk. Good for secrets or temporary processing.

5. Networking Modes

Bridge (Default): Containers sit on a private internal network (docker0). They talk to each other via IP and to the outside via NAT. Requires port mapping (`-p 80:80`) to be accessible externally.

Host: Container removes network isolation and shares the Host's networking namespace directly. Faster (no NAT), but port conflicts can occur.

None: No networking. Container has localhost only. Secure isolation.

Overlay: Used in Docker Swarm/Multi-node setups. Allows containers on different physical hosts to communicate.

Macvlan: Assigns a MAC address to the container, making it appear as a physical device on the network. Used for legacy apps monitoring network traffic.

6. Image Optimization

Layer Caching: Docker caches lines in a Dockerfile. Order matters! Put `COPY requirements.txt` and `RUN pip install` *before* `COPY . .`. This way, changing code doesn't force a re-install of dependencies.

.dockerignore: Just like `.gitignore`. Prevents large or sensitive files (`.git`, `node_modules`, secrets) from being sent to the build context.

Pruning (Cleanup):

```
docker system prune -a: Remove all stopped containers, unused networks, and dangling images (caution!).
```

7. Docker Compose

Tool for defining and running multi-container Docker applications via `docker-compose.yml`.

Command	Description
<code>docker-compose up -d</code>	Start all services in background.
<code>docker-compose down</code>	Stop and remove containers/networks.
<code>docker-compose build</code>	Rebuild images.
<code>docker-compose logs -f</code>	View logs for all services.
<code>docker-compose ps</code>	List services status.